FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES

CS054
(INTELLIGENT SYSTEMS)

EXERCISE
2

Sentiment Analysis

Name:	Eric V. Tividad Jr.
Section:	AN 42
Professor:	Ms. Crisola Tan

I. PROGRAM OUTCOME/S (PO) ADDRESSED BY THE LABORATORY EXERCISE

- Design and evaluate solutions for computing problems. [PO: e]

II. COURSE LEARNING OUTCOME/S (CLO) ADDRESSED BY THE LABORATORY EXERCISE

- To design an Intelligent System based on methods learned. [CLO: 3]

III. INTENDED LEARNING OUTCOME/S (ILO) OF THE LABORATORY EXERCISE

At the end of this exercise, students must be able to:

- Demonstrate understanding of Sentiment Analysis.
- Design and develop a simple program about Sentiment Analysis using Python.

IV. BACKGROUND INFORMATION

Sentiment Analysis can assist us in determining the mood and feelings of the general public as well as obtaining useful information about the setting. Sentiment Analysis is the process of assessing data and categorizing it according to the research's needs.

Sentiments can be used to gain a better understanding of different events and their consequences. It goes by various names, including "sentiment analysis," "opinion mining," "opinion extraction," "sentiment mining," "subjectivity analysis," "affect analysis," "emotion analysis," and "review mining," but they all serve the same objective. We might be able to figure out what people like or want by evaluating their feelings.



TextBlob

TextBlob is a python library and offers a simple API to access its methods and perform basic NLP tasks. Textblob's sentiment function returns polarity and subjectivity as two characteristics.

The polarity ranges from [-1,1], with 1 denoting a positive statement and -1 denoting a negative one. Subjective statements usually refer to personal feelings, emotions, or judgments, whereas objective sentences refer to facts.

Subjectivity is also a float with a value between
0 and 1.

V. GRADING SYSTEM / RUBRIC (please see separate sheet)

VI. LABORATORY ACTIVITY

Perform the following:

1. Add the following code:

```
from textblob import TextBlob
```

2. Declare a variable that will accept a sentence:

```
sentence = input("Ënter a sentence : ")
```

3. Next initialize TextBlob

```
s = TextBlob(sentence)
```

4. Then check the polarity of the sentence by using the code below:

```
p = s.sentiment.polarity
```

5. Make a decision based on the polarity. If the polarity of the sentence is less than zero the sentiment is negative. If the polarity is equals to zero, then the sentiment is neutral. If the polarity is greater than zero, the sentiment is positive.

```
#Negative p<0, Neutral p=0, Positive
p>0 if p <0:
print("Negative") elif
p==0:
print("Neutral") elif p>0
and p<=1:
    print("Positive")
```

6. Make the program perform the sentiment analysis 10 times and make a summary of how many sentences are positive, neutral or negative.

OUTPUT:

```
Enter an item: hello
Enter an item: amog
Enter an item: smog
Enter an item: dragon
Enter an item: bomb
Enter an item: sandwich
Enter an item: oop
Enter an item: yes
Enter an item: i am evil
Neutral
Neutral
Neutral
Neutral
Neutral
Neutral
Neutral
Neutral
Negative
```

CODE:

```
 from textblob import TextBlob

sentences = []
for i in range(9):
    sentences.append(input("Enter an item: "))

for sentence in sentences:
    s = TextBlob(sentence)
    p = s.sentiment.polarity

    # Negative p<0, Neutral p=0, Positive p>0 if p <0:
    if p <0:
        print("Negative")

    elif p==0:
        print("Neutral")

    elif p>0 and p<=1:
        print("Positive")
```

CODE SCREENSHOT:

u

```
from textblob import TextBlob

sentences = []
for i in range(9):
    sentences.append(input("Enter an item: "))

for sentence in sentences:
    s = TextBlob(sentence)
    p = s.sentiment.polarity

    # Negative p<0, Neutral p=0, Positive p>0 if p <0:
    if p <0:
        print("Negative")

    elif p==0:
        print("Neutral")

    elif p>0 and p<=1:
        print("Positive")
```

VII. CHALLENGE

Write a program that will ask the user for a string containing a sentence. The program will then compute the happiness and sadness level of the sentence using the two functions described below. If the happiness level is higher than sadness level, then the tone of the sentence is happy. If the sadness level is higher, then the tone of the sentence is sad. Otherwise, it is neutral.

Find and print the tone of the sentence by printing the number of happy words followed by the number of sad words, followed by a simple statement of the analysis (whether positive, negative or neutral).

To accomplish this, you will write a function called number_happy(sentence) which returns the number of words in a given string called “sentence” that are happy. To do this, look for a public dataset containing a list of happy words. Find the total number of words in “sentence” that are in the happy words dataset.

Next, write a second function called number_sad(sentence) that works the same way but instead counts the number of sad words, also from a separate dataset.

OUTPUT:

```
Enter a sentence: i am just depressed today.
Number of happy words: 0
Number of sad words: 1
Tone: Negative
```

CODE:

```
def challenge():
    happy_words = set()
    sad_words = set()

    # gets the text file from the root folder
    dataset_path = (
        Path(__file__).resolve().parent/"NRC-Emotion-Lexicon-Wordlevel-v0.92.txt")

    # translates into a dataset the .txt file
    with open(dataset_path, "r", encoding="utf-8") as file:
        reader = csv.reader(file, delimiter="\t")

        for row in reader:
            if len(row) == 3:
                word = row[0].lower()
                emotion = row[1].lower()
                association = row[2].strip()

                if association == "1":
                    if emotion == "joy":
                        happy_words.add(word)
                    elif emotion == "sadness":
                        sad_words.add(word)
```

```
    # cleans and also counts how many happy words are in a sentence
    def number_happy(sentence):
        words = sentence.lower().split()
        count = 0

        for word in words:
            word = word.strip(string.punctuation)

            if word in happy_words:
                count += 1

        return count

    # same logic but with sadness :(
    def number_sad(sentence):
        words = sentence.lower().split()
        count = 0

        for word in words:
            word = word.strip(string.punctuation)

            if word in sad_words:
                count += 1

        return count

    sentence = input("Enter a sentence: ")

    happy_count = number_happy(sentence)
    sad_count = number_sad(sentence)

    print("Number of happy words:", happy_count)
    print("Number of sad words:", sad_count)

    if happy_count > sad_count:
        print("Tone: Positive")

    elif sad_count > happy_count:
        print("Tone: Negative")

    else:
        print("Tone: Neutral")

if __name__ == "__main__":

    # Just remove the comment on of these to try each of the activity
    #first_activity()
    challenge()
```

CODE SCREENSHOT:

```
25  def challenge():
26      happy_words = set()
27      sad_words = set()
28
29      # gets the text file from the root folder
30      dataset_path = (
31          Path(__file__).resolve().parent/"NRC-Emotion-Lexicon-Wordlevel-v0.92.txt")
32
33      # translates into a dataset the .txt file
34      with open(dataset_path, "r", encoding="utf-8") as file:
35          reader = csv.reader(file, delimiter="\t")
36
37          for row in reader:
38              if len(row) == 3:
39                  word = row[0].lower()
40                  emotion = row[1].lower()
41                  association = row[2].strip()
42
43                  if association == "1":
44                      if emotion == "joy":
45                          happy_words.add(word)
46                      elif emotion == "sadness":
47                          sad_words.add(word)
48
49      # cleans and also counts how many happy words are in a sentence
50      def number_happy(sentence):
51          words = sentence.lower().split()
52          count = 0
53
54          for word in words:
55              word = word.strip(string.punctuation)
56
57              if word in happy_words:
58                  count += 1
59
60          return count
```

```python
def challenge():
    # same logic but with sadness :(
    def number_sad(sentence):
        words = sentence.lower().split()
        count = 0

        for word in words:
            word = word.strip(string.punctuation)

            if word in sad_words:
                count += 1

        return count

    sentence = input("Enter a sentence: ")

    happy_count = number_happy(sentence)
    sad_count = number_sad(sentence)

    print("Number of happy words:", happy_count)
    print("Number of sad words:", sad_count)

    if happy_count > sad_count:
        print("Tone: Positive")

    elif sad_count > happy_count:
        print("Tone: Negative")

    else:
        print("Tone: Neutral")

if __name__ == "__main__":

    # Just remove the comment on of these to try each of the activity
    #first_activity()
    challenge()
```

VIII. REFERENCES

- Artasanchez A. Joshi P.(2020). Artificial Intelligence with Python: You Complete Guide to Building Intelligent apps using Python 3.x 2nd Edition. Packt Publishing
- Attaway, Stormy (2013). MATLAB ®. A Practical Introduction to Programming and Problem Solving Third Edition. Elsevier, Inc.

Online Sources:

- https://www.cuelogic.com/blog/role-of-python-in-artificial-intelligencehttps://monkeylearn.com/sentimentanalysis
- https://towardsdatascience.com/a-beginners-guide-to-sentiment-analysis-in-python-95e354ea84f6
- https://research.google.com/colaboratory
- https://www.youtube.com/watch?v=RLYoEyIHL6A
